



Python & Pandas: The Data Hero's Journey

คู่มือเริ่มต้นฉบับละเอียด: จัดการและวิเคราะห์ข้อมูลแบบ Step-by-Step



PLAYER

Beginner Data Scientist
(นักวิทยาศาสตร์ข้อมูลมือใหม่)

PRESS START



WEAPON

Google
Colaboratory



MISSION

เปลี่ยนข้อมูลดิบให้เป็น
บุคลิก (Insights)

Level 0: The Setup (เตรียมอาวุธ)

THE MISSION (ภารกิจ)



ภารกิจ: ติดตั้งและเรียกใช้งาน Pandas เพื่อเตรียมพร้อมสำหรับการจัดการข้อมูล

Step 1: โดยปกติ Google Colab จะมี Pandas มาให้อยู่แล้ว แต่ถ้ารันในเครื่องตัวเองต้องใช้คำสั่ง 'pip install'

Step 2: เรียกใช้งาน Library และตั้งชื่อเล่นว่า 'pd' (เป็นมาตรฐานสากลที่โปรทั่วโลกใช้กัน)



THE CHEAT CODE (สูตรโกง)

Terminal - Colab

```
# ติดตั้ง Library (ถ้าจำเป็น)
!pip install pandas

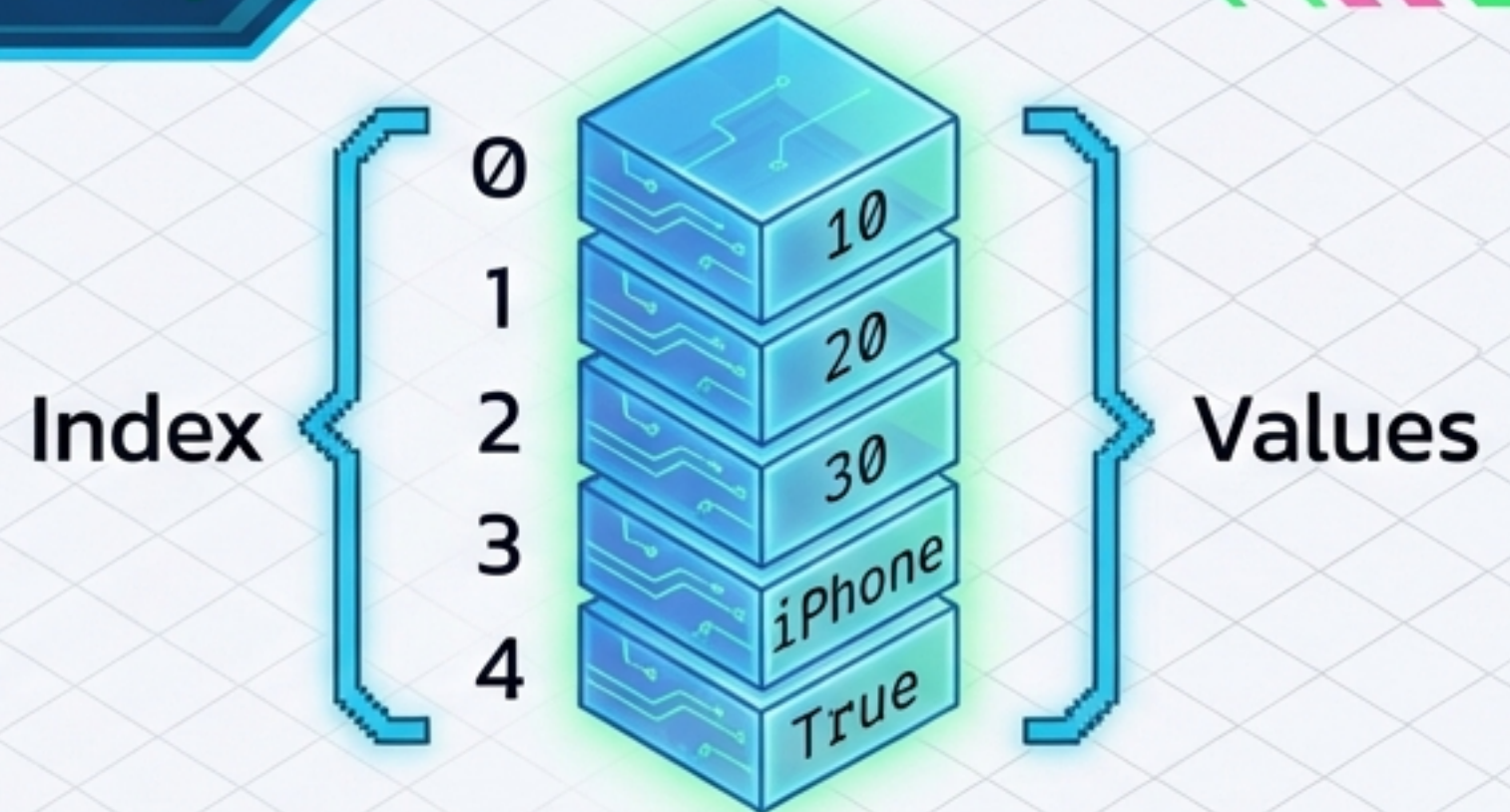
# เรียกใช้งาน (Equip)
import pandas as pd
import numpy as np
```



Level 1: The Foundation – Series (ซีรีส์)

Concept: 1D Array

Series คือโครงสร้างข้อมูลแบบ 1 มิติ (1D Array) เหมือน 'คอลัมน์เดียว' หรือ 'แถวยาวๆ' มี 2 ส่วนคือ **Index** (ป้ายกำกับตำแหน่ง) และ **Values** (ค่าข้อมูลจริง)



Terminal – Colab Code

```
# สร้าง Series จาก List
data = [10, 20, 30, "iPhone", True]
ps = pd.Series(data)
print(ps)
```

Loot

```
0    10
1    20
2    30
3  iPhone
4    True
dtype: object
```


Level I: Customization (กำหนด Index เอง)



Power Up: เราไม่จำเป็นต้องใช้ Index 0, 1, 2 เสมอไป! เราสามารถกำหนด Index เองได้เพื่อให้สื่อความหมาย (เช่น ชื่อสินค้า หรือ วันที่)

ข้อควรระวัง: จำนวน Index ที่กำหนด ต้องเท่ากับจำนวนข้อมูลที่มี

THE CHEAT CODE (สูตรโกง)

Terminal - Colab

```
# สร้าง Series จาก List พร้อมกำหนด Index  
items = ["มะม่วง", "กล้วย", "องุ่น"]  
idx = [50, 20, 30] # สมมติเป็นราคา
```

```
# สร้าง Series แบบกำหนด Index  
ps_custom = pd.Series(items, index=idx)  
print(ps_custom)
```

```
50    มะม่วง  
20    กล้วย  
30    องุ่น  
dtype: object
```

 **Loot**

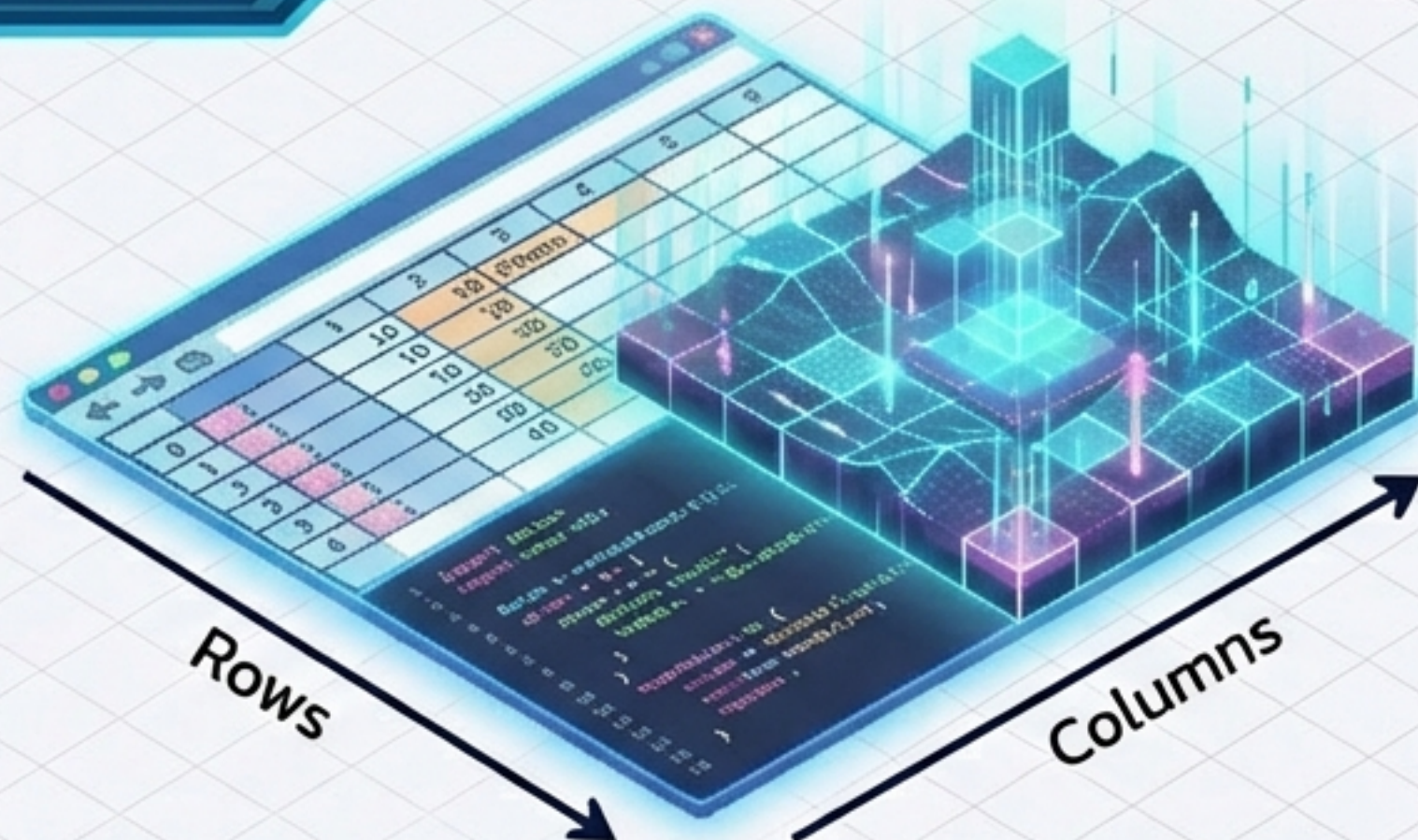


READY TO CODE

Level 2: The Grid – DataFrame (ดาต้าเฟรม)

Concept: DataFrame

DataFrame คือโครงสร้างข้อมูลแบบ 2 มิติ (2D Array) เหมือนตาราง Excel เป๊ะ!
ประกอบด้วย **Rows** (แถว)
และ **Columns** (คอลัมน์)



Crafting the Grid

```
# สร้าง DataFrame จาก List
data = [10, 20, 30, 40]
df = pd.DataFrame(data)
print(df)
```

	0
0	10
1	20
2	30
3	40

🔑 Loot



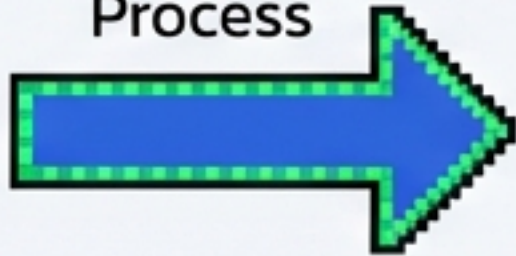
Level 2: Blueprinting (สร้างจาก Dictionary)

****Key**** = ชื่อคอลัมน์
(Header)



Dictionary Book

Mapping
Process



Data Table

****Value**** = ข้อมูลในแถว
(List)

****The Best Way:****
การสร้างจาก Dictionary
เป็นวิธีที่นิยมที่สุดและอ่านง่าย

Terminal - Colab

```
products = {  
    "Name": ["Keyboard", "Mouse"],  
    "Price": [4000, 500],  
    "Category": ["IT", "IT"]  
}  
df = pd.DataFrame(products)  
print(df)
```

Loot

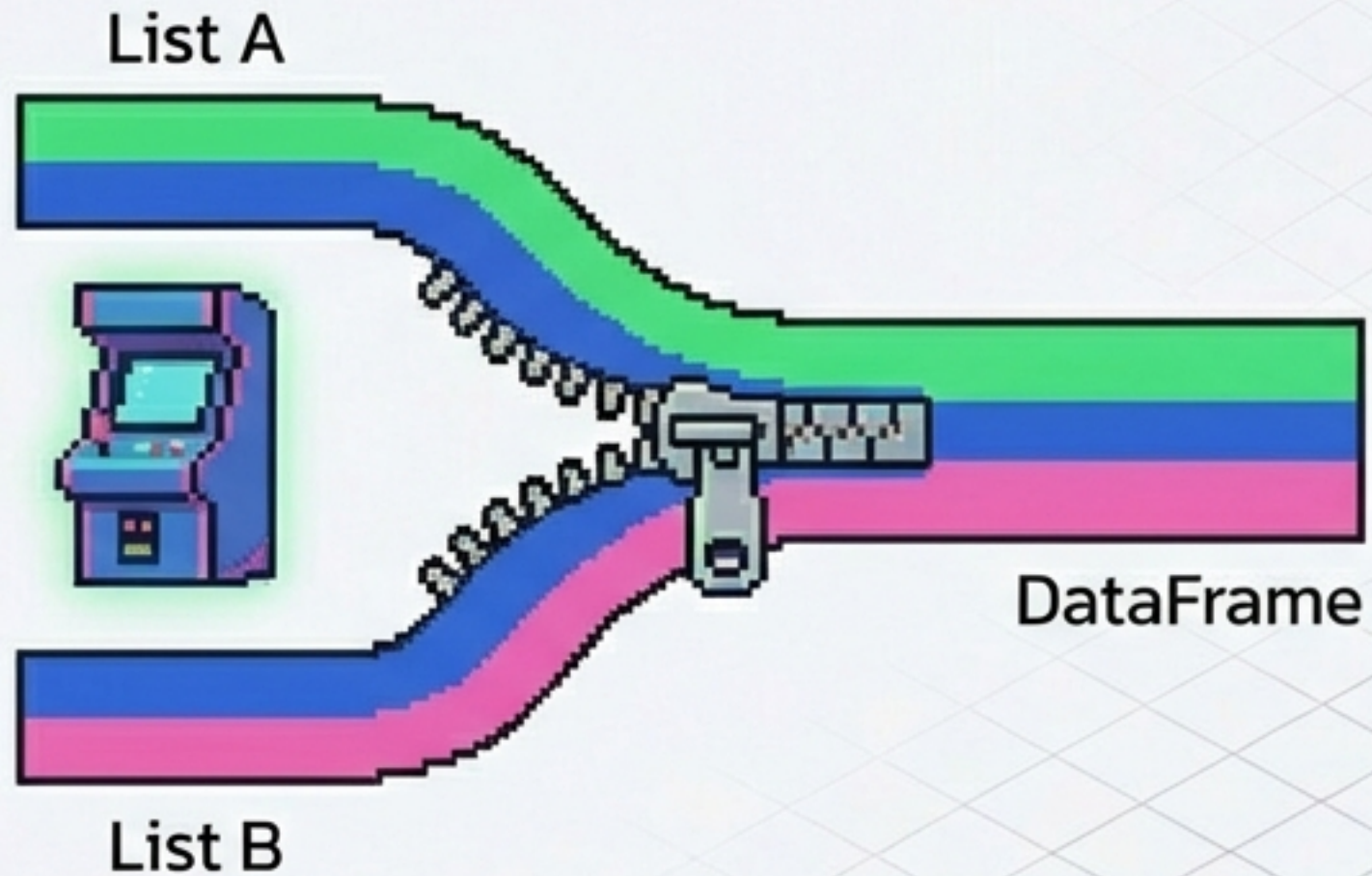


	Name	Price	Category
0	"Keyboard"	4000	"IT"
1	"Mouse"	500	"IT"

READY TO CODE

Level 2: Combo Move (Zip & Column Names)

Strategy



Technique: ใช้ `zip()` จับคู่ข้อมูลแยก List แล้วกำหนดชื่อ `columns` ตอนสร้าง DataFrame เพื่อให้ตารางสมบูรณ์

Execution

Terminal - Colab

```
name = ["Keyboard", "Mouse"]
price = [4000, 500]

# Zip รวบรวม และตั้งชื่อคอลัมน์
datas = list(zip(name, price))
cols = ["Product Name", "Price"]

df_zip = pd.DataFrame(datas, columns=cols)
print(df_zip)
```

	Product Name	Price
0	Keyboard	4000
1	Mouse	500

Loot



READY TO CODE

Level 3: Save Game (Export to CSV)



The Command

``to_csv`` ใช้ส่งออกข้อมูลเป็นไฟล์ .csv

Pro Tip

ปกติ Pandas จะแถมเลข Index มาด้วย
ถ้าไม่ต้องการให้ใส่ ``index=False``

Encoding

ถ้าข้อมูลมีภาษาไทย ให้เพิ่ม
``encoding='utf-8`` กันภาษาต่างดาว

Terminal - Colab

```
# บันทึกไฟล์ CSV
df.to_csv("products.csv", index=False)

# กรณีมีภาษาไทย
df.to_csv("products_th.csv", encoding="utf-8")
```

READY TO CODE



Level 3: Load Game (Import CSV)

Loading...

Mission:

นำข้อมูลจากภายนอกเข้ามาวิเคราะห์ใน Colab

Command:

`pd.read\_csv()` คือคำสั่งที่ Data Scientist ใช้บ่อยที่สุดในโลก!



****Note:****

อย่าลืมอัปโหลดไฟล์ขึ้น Google Colab ก่อนเรียกใช้งาน



Terminal - Colab

```
# อ่านไฟล์ CSV
df = pd.read_csv("products.csv")

# อ่านไฟล์ที่มีภาษาไทย
df_th = pd.read_csv("products_th.csv", encoding="utf-8")
print(df_th.head()) # ดูตัวอย่างข้อมูล
```

READY TO CODE

Level 3: Advanced Loading (Excel & Options)



Excel Support

- Pandas อ่านไฟล์ .xlsx ได้เลยด้วย
`read\_excel`
- Selective Loading
 - usecols: เลือกโหลดเฉพาะคอลัมน์ที่ต้องการ (ประหยัดแรม)
 - index_col: กำหนดคอลัมน์ที่จะใช้เป็น Index แทนที่ที่โหลด

Terminal - Colab

```
# อ่าน Excel เลือกเฉพาะบางคอลัมน์ และตั้งชื่อสินค้าเป็น Index  
  
df = pd.read_excel("stock.xlsx",  
                  usecols=["Name", "Price"],  
                  index_col="Name")  
  
print(df)
```

READY TO CODE

Level 4: Scouting (ส่องดูข้อมูลเบื้องต้น)

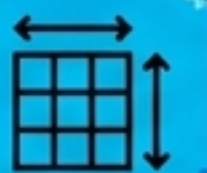


ID	Name	Score	Date	Category
001	สมิธ	95	2556-01-01	0
001	สมิธ	95	2556-01-01	0
003	ประจักษ์	98	2556-01-01	0
004	ประจักษ์	98	2556-01-01	0
005	ประจักษ์	98	2556-01-01	4



Head/Tail

`head(n) / tail(n)` :
ดูหัวตารางและท้ายตาราง



Shape

`shape` : ดูขนาด (จำนวนแถว,
จำนวนคอลัมน์)



Describe

`describe()` : สรุปค่าทางสถิติ
(ค่าเฉลี่ย, สูงสุด, ต่ำสุด)
ของข้อมูลตัวเลข

Terminal - Colab

```
print(df.head(5))    # ดู 5 แถวแรก  
print(df.shape)      # (Rows, Columns)  
print(df.describe()) # ดูสถิติ
```

READY TO CODE

Level 4: Tactical Targeting (Slicing)



0	0	0	0	anw	hot
1	1	2	10	2.21	30.0
2	2	3	10	2.22	25.0
3	3	3	11	2.21	25.0
4	4	3	11	3.22	20.0



Mission: เราจะเลือกข้อมูลเฉพาะช่วงที่ต้องการ (Slicing)



Syntax: ใช้เครื่องหมาย Colon : `ระบุ`
`[จุดเริ่ม : จุดสิ้นสุด]`



Rule: จุดสิ้นสุดจะไม่ถูกรวมมาด้วย (Stop - 1)
เช่น `1:4` จะได้แถวที่ 1, 2, 3

Terminal - Colab

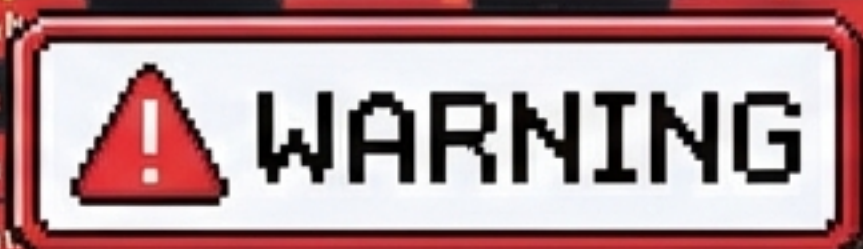
```
# เลือกแถวที่ 1 ถึงก่อนแถวที่ 4
subset = df[1:4]

# เลือกตั้งแต่แถวแรก ถึงก่อนแถวที่ 3
subset_top = df[:3]
print(subset)
```

READY TO CODE

BOSS FIGHT: The Glitch (จัดการค่าว่าง)

			...	...	
0	NaN	NaN	NaN	NaN	NaN
1	NaN				NaN
2	NaN				NaN
3	NaN				NaN
4	NaN				NaN
5	NaN	NaN	NaN	NaN	NaN
6	NaN	NaN	NaN	NaN	NaN



Strategy



The Enemy: ข้อมูลที่ไม่สมบูรณ์ (Missing Values) จะแสดงเป็น `NaN`



Detection: ใช้ `isnull().sum()`
เช็คว่ามีค่าว่างกี่จุด



Attack 1 (Delete): ถ้าข้อมูลหายเยอะจนซ่อมไม่ได้ ให้ลบทิ้งด้วย `dropna()`

Terminal - Battle Code

```
# เช็คค่าว่าง
print(df.isnull().sum())

# ลบแถวที่มีค่าว่างทิ้งทั้งหมด
df_clean = df.dropna()
```

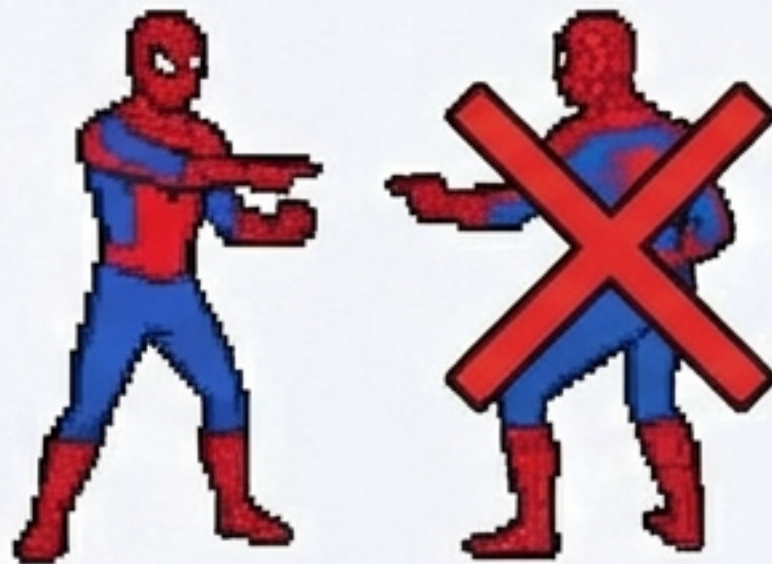
READY TO CODE

BOSS FIGHT: Patching & Clones (เติมค่า & ลบตัวซ้ำ)



Attack 2 (Repair):

ไม่อยากลบ? ใช้ `fillna()`
เติมค่าลงไปแทน
(เช่น เติม 0 หรือค่าเฉลี่ย)



Attack 3 (Dedup):

ข้อมูลซ้ำซ้อนทำให้ผล
วิเคราะห์เพี้ยน กำจัดด้วย
`drop_duplicates()`

Terminal - Battle Code

```
# เติบค่าว่างด้วย 0  
df.fillna(0, inplace=True)  
  
# ลบข้อมูลซ้ำ (เก็บตัวแรกไว้)  
df.drop_duplicates(inplace=True)
```

READY TO CODE



YOU WIN! (ยินดีด้วย คุณผ่านด่านพื้นฐานแล้ว)



Next Mission: ลองนำ Dataset ของจริงมาเล่นใน Google Colab เลย!

Keep Coding & Have Fun!